

How the Residual-Aware Forecasting Pipeline Is Built

A code walkthrough of `SujatAhmed/498R-kdd`

Companion to *One Step Closer to Ground Truth: A Multi-Scale Residual-Aware Representation Learning Pipeline for Predicting Time Series Data*
(Biswas et al., KDD '26, arXiv:2606.10678v2)

Base code: `iTransformer` (thuml, ICLR '24) · Python / PyTorch 2.1 · ~3.9k lines, 30 .py files, 48 shell scripts

Contents

1	Summary	1
2	The idea in the paper	2
3	Repository map	3
4	End-to-end workflow	4
5	Stage 1: training the base model	5
5.1	Data loading and splits	5
5.2	The <code>iTransformer</code> network	5
5.3	Training loop and what it saves	6
6	Stages 2–3: building the residual series	6
6.1	<code>save_as_csv.py</code> : merging overlapping windows (Eq. 2)	6
6.2	<code>flactuation_calculation.py</code> : residuals (Eq. 3)	6
7	Stage 4: training the meta-corrector	7
8	Stage 5: correction and scoring	7
8.1	Alignment	7
8.2	Correction	7
8.3	Re-scaling for the reported numbers	7
9	Where the code and the paper differ	8
10	How to run it (ECL, $H = 96$)	9
11	Inherited <code>iTransformer</code> code	10

1 Summary

The repository is a fork of the official **iTransformer** codebase with four changes: it saves predictions for every split, and it adds three stand-alone post-processing scripts. Together these implement

the paper’s two-stage idea: train a forecaster, turn its errors into a new time series, train a second forecaster on those errors, and subtract a small multiple of the predicted error from the first forecast.

The pipeline is **manual**. There is no single entry point. You run `run.py` twice, with three scripts in between, and fill in file paths by hand at the top of each script.

- Both stages use the same unchanged iTransformer network (`model/iTransformer.py`), which is why the paper reports $2 \times 5.47\text{M} \approx 10.9\text{M}$ parameters.
- The novel code is small: `save_as_csv.py` (averages overlapping window forecasts, paper Eq. 4), `fluctuation_calculation.py` (builds the residual series, Eq. 5) and `final_performnace.py` (applies $\hat{Y} - \alpha E$, Eq. 7, with $\alpha = 0.05$).
- **High:** The Huber loss the paper uses for the meta-corrector is **not in the code**. `--loss` is parsed but never read, and `_select_criterion()` always returns `nn.MSELoss()`.
- **High:** The “scaled” MSE/MAE that `final_performnace.py` prints uses a scaler fit on the *test ground truth*, pooled across all channels. The baselines use per-channel scalers fit on the training split. The two numbers measure different things, and this matters for the Table 1 comparison.
- **Medium:** The autoregressive test-time procedure for the meta-corrector (paper §3.2.4, Fig. 2h) is not in the active code. At test time the meta-model reads the true past residuals, one sliding window at a time.

2 The idea in the paper

Take a multivariate series with C channels. A lookback window $X \in \mathbb{R}^{C \times L}$ (here $L = 96$) is used to predict the next $H \in \{96, 192, 336, 720\}$ steps. The **base model** f_θ is a normal single-stage forecaster:

$$\hat{Y}_{L+1:L+H} = f_\theta(X_{1:L}). \quad (1)$$

Windows slide with stride 1, so each time index t is predicted by up to H different windows. The paper merges these into one series P by averaging:

$$P_t = \frac{1}{|\mathcal{I}_t|} \sum_{i \in \mathcal{I}_t} \hat{y}_t^{(i)}, \quad (2)$$

where $\mathcal{I}_t$ is the set of windows whose horizon covers t . The **residual series** is the forecast minus the truth. It has no values for the first L steps, because no window predicts them:

$$R = P - Y_{L+1:T}. \quad (3)$$

A second model, the **meta-corrector** f_ϕ , has the same architecture and is trained on R exactly as if R were an ordinary dataset. It predicts future residuals $E = f_\phi(R_{1:L})$. The final forecast removes a shrunk copy of the predicted error:

$$\hat{Y}_i^{\text{final}} = f_\theta(X_i) - \alpha f_\phi(R_i), \quad \alpha = 0.05. \quad (4)$$

The paper’s justification (Appendix A) is short. Since $y - (\hat{y} - \alpha E) = -(R - \alpha E)$, the correction helps exactly when

$$\|R - \alpha f_\phi(R)\|^2 < \|R\|^2, \quad (5)$$

that is, when the meta-model predicts the residual’s sign and size well enough. The paper tests this with Ljung–Box tests (residuals are not white noise, $p < 10^{-6}$) and counterfactuals: meta-models trained on shuffled or Gaussian-noise residuals make the final forecast worse, not better. α plays the role of shrinkage in gradient boosting. The paper chose it by grid search over $[0, 1]$ in steps of 0.05, counting wins across the 32 dataset $\times$ horizon settings.

Two further design choices from §3.3 show up in the code:

- **Huber loss for the meta-model.** Residuals change sign often, and Huber’s linear tail gives bounded gradients when the predicted sign is wrong:

$$\mathcal{L}_\delta(a) = \begin{cases} \frac{1}{2}a^2, & |a| \leq \delta, \\ \delta(|a| - \frac{1}{2}\delta), & |a| > \delta. \end{cases}$$

- **Correct in native units.** Both models standardise their inputs internally, but outputs are inverse-transformed to raw units before the subtraction. Only then is the result re-scaled for scoring.

3 Repository map

Files marked PIPELINE are the ones that matter for the paper. Files marked INHERITED come from upstream iTransformer and are unused by, or incidental to, this method.

Path	Role	What it does
run.py	PIPELINE	CLI entry point. Parses ~45 arguments, fixes seed 2023, builds a <code>setting</code> string used to name checkpoint and result folders, then calls <code>exp.train()</code> and <code>exp.test()</code> . Used for <i>both</i> stages.
experiments/exp_long_term_forecasting.py	PIPELINE	Training, validation and test loops. Modified from upstream so it writes predictions for train, val and test, in real units, to <code>./predictions/<model_id>/</code> , and computes test metrics in real units. Also contains two commented-out autoregressive <code>test()</code> variants.
experiments/exp_basic.py	PIPELINE	Base class. Picks the device and maps <code>--model</code> names to model classes.
data_provider/data_factory.py	PIPELINE	Builds the <code>Dataset</code> and <code>DataLoader</code> . Modified: <code>shuffle=False</code> and <code>drop_last=False</code> for every split, so saved predictions stay in time order and cover every window.
data_provider/data_loader.py	PIPELINE	Dataset classes: ETT hour/minute (fixed 12/4/4-month split), Custom (70/10/20 split, used for ECL, Traffic, Weather, Exchange and the residual CSVs), plus Solar, PEMS and Pred. Standardises each channel with a scaler fit on the train split.
model/iTransformer.py	PIPELINE	The network used for both f_θ and f_ϕ . Unchanged from upstream.
layers/*.py	PIPELINE	Inverted embedding, encoder layers and attention used by iTransformer. They also hold Prob/Flow/Flash/Reformer attention for the other models.
save_as_csv.py	PIPELINE	Stage 2. Loads one split’s (N, H, C) prediction array and averages overlapping windows into one $(N + H - 1, C)$ series. Writes <code>{split}_predictions.csv</code> with the dataset’s column names.
flactuation_calculation.py	PIPELINE	Stage 3. Concatenates the train/val/test CSVs, aligns them with ground truth from index 96, computes <code>pred - truth</code> , and writes a residual CSV with the original <code>date</code> column.
final_performnace.py	PIPELINE	Stage 5. Aligns base and residual test predictions, applies <code>base - α · residual</code> , prints raw-scale MSE/MAE before and after, then prints “scaled” metrics.
utils/*.py	PIPELINE	MSE/MAE/RMSE/MAPE/MSPE, early stopping, LR schedule (<code>type1</code> halves each epoch), plotting, and calendar features.
scripts/multivariate_forecasting/*	PIPELINE	Shell scripts with each dataset’s hyper-parameters for the four horizons. The README tells you to use these for both stages.
model/ (other 9 models)	INHERITED	Transformer, Informer, Reformer, Flowformer and Flashformer, plus their inverted “i” versions. Selectable with <code>--model</code> , but the paper uses only iTransformer.
experiments/exp_long_term_forecasting_partial.py	INHERITED	iTransformer’s “train on some variates, test on all” experiment (<code>--exp_name partial_train</code>). It does not save predictions.
scripts/ (4 other families)	INHERITED	<code>boost_performance</code> , <code>variate_generalization</code> , <code>increasing_lookback</code> , <code>model_efficiency</code> : scripts for upstream iTransformer ablations. They are not part of this paper.

4 End-to-end workflow

Every hand-off between stages goes through a file on disk. Stages 1 and 4 run the same code with different data (and, per the paper, a different loss).

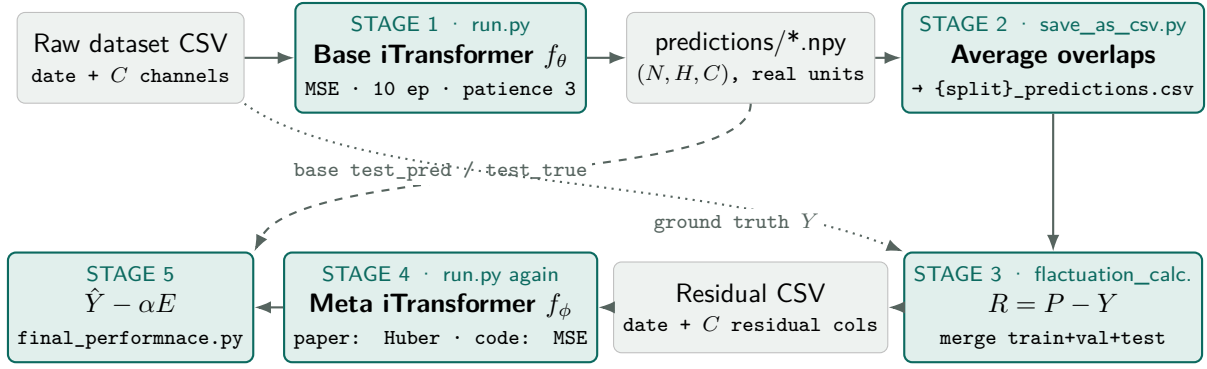


Figure 1: Solid arrows are the main data flow. The dashed line carries the base model’s test forecasts, which skip the residual stages and go straight to the final combination. The dotted line is the raw ground truth used to form residuals.

5 Stage 1: training the base model

5.1 Data loading and splits

`data_provider/data_factory.py` maps `--data` to a dataset class. For the custom datasets (ECL, Traffic, Weather, Exchange), `Dataset_Custom` in `data_loader.py` does the following:

1. Reorders columns to `[date, ...features, target]`. With `--features M` the target is just another channel.
2. Splits in time order, 70 / 10 / 20 %. Val and test start L rows early so their first window has a full lookback: `border1s = [0, n_train - L, T - n_test - L]`.
3. Fits `sklearn.StandardScaler` on the *train* rows only, per channel, and transforms the whole series.
4. Builds calendar features (`timeF`: month, day, weekday, hour and so on, scaled to $[-0.5, 0.5]$).

Each item is a window: `seq_x = data[i : i+L]`, `seq_y = data[i+L-label_len : i+L+H]`. There are $N = n - L - H + 1$ windows in a split of n rows. For ECL ($T = 26,304$, $C = 321$, $H = 96$), the train split has 18,412 rows and 18,221 windows.

The ETT datasets use fixed borders of 12/4/4 months ($\times 4$ for the 15-minute files) and ignore any rows after month 20.

5.2 The iTransformer network

`model/iTransformer.py` “inverts” the usual Transformer. Each **channel** becomes a token, not each time step:

1. **Instance normalisation** (`use_norm`): subtract each window’s per-channel mean and divide by its std. The statistics are added back at the end.
2. **Embedding** (`DataEmbedding_inverted`): permute $B \times L \times N \rightarrow B \times N \times L$, append the time-feature rows as extra tokens, then apply one `nn.Linear(L, d_model)`. Each channel’s whole 96-step history becomes one vector.
3. **Encoder**: `e_layers` blocks of full multi-head self-attention *across channels*, then a position-wise FFN (two 1×1 convs). Each block uses post-LayerNorm and residual connections, with a final LayerNorm. Attention cost is $O(N^2)$ in the number of channels.
4. **Projection**: `nn.Linear(d_model, H)` per token, permute back to $B \times H \times N$, and drop the time-feature tokens.

There is no decoder. The `dec_inp` tensor that the experiment loop builds is passed in but ignored.

5.3 Training loop and what it saves

`Exp_Long_Term_Forecast.train()` uses Adam, the loss from `_select_criterion()`, and learning-rate schedule type1 ($lr_e = lr_0 \cdot 0.5^{e-1}$). Early stopping watches validation loss and keeps the best weights in `checkpoints/<setting>/checkpoint.pth`. Compared with upstream iTransformer, the fork adds `_save_predictions_real_scale()`, which inverse-transforms arrays to raw units and saves them:

File (in <code>predictions/<model_id>/</code>)	When written	Shape
<code>epoch{e}_train_pred_{H}.numpy / _true_</code>	End of every epoch, from the forward passes made <i>during</i> training	(N_{train}, H, C)
<code>epoch{e}_val_pred_{H}.numpy / _true_</code>	Every epoch, in eval mode	(N_{val}, H, C)
<code>test_pred_{H}.numpy / test_true_{H}.numpy</code>	Once, in <code>test()</code> , with the best checkpoint loaded	(N_{test}, H, C)

`test()` also writes `results/<setting>/{pred,true,metrics}.numpy` and appends to `result_long_term_forecast.txt`. Note that these metrics are now in **raw units**. Upstream iTransformer reports them in standardised units, so the printed numbers are not comparable to published iTransformer tables.

6 Stages 2–3: building the residual series

6.1 `save_as_csv.py`: merging overlapping windows (Eq. 2)

Window i predicts global offsets $i, \dots, i + H - 1$ relative to the start of the split’s targets. The script adds each prediction into `index_sum[i*stride + j]`, counts contributions, and divides:

```
for i in range(N):                # window
    for j in range(pred_len):      # step in horizon
        index_sum[i*stride + j] += predictions[i, j, :]
        index_count[i*stride + j] += 1
final = index_sum / index_count    # shape (N + H - 1, C)
```

Interior points are averaged over H forecasts. Points near the ends have fewer. Because $N + H - 1 = n - L$, each split’s merged series covers exactly its target rows. Train covers rows $[L, n_{\text{train}})$, val covers $[n_{\text{train}}, n_{\text{train}} + n_{\text{val}})$, and test covers the rest. So the three CSVs join end to end with no gaps or overlaps. You run the script once per split and fill in `dataset_key`, `split`, `features (C)`, `preds_filename`, `folder_path` and the `file_paths` dictionary, which is used only to get column names.

6.2 `fluctuation_calculation.py`: residuals (Eq. 3)

1. Finds the files in `directory_path` whose names contain “train”, “vali” and “test”, and concatenates them into `{dataset}_merged_predictions.csv`.
2. Slices the ground-truth CSV and the merged predictions to rows `[start, end)`, by default `[96, 20000)`. The slice is applied to both files by the same helper.
3. If the prediction series is shorter than the truth, it pads it with the mean of its last 125 rows.
4. Computes `fluctuation = df2 - df1`, that is, **prediction** – **truth**, matching $R = P - Y$. It then puts the `date` column back and writes `{dataset}.csv`.

Why the sign matters

Because R is defined as prediction $-$ truth, a positive predicted residual means “the base model will overshoot”. That is why stage 5 *subtracts* αE . If you flip the subtraction in stage 3, you must also flip the sign in stage 5.

7 Stage 4: training the meta-corrector

There is no separate meta-model code. The README says to put the residual CSV in the dataset folder and run `run.py` again, pointing `--root_path/--data_path` at it. It becomes one more custom dataset:

- It gets its own 70/10/20 split of the shorter series (length $T - L$), and its own per-channel scaler fit on residual-train rows. The base model’s weights and statistics are never used, which is the paper’s “no shared parameters” protection against leakage.
- Inputs and targets are both residual windows: $R_{t-L:t} \mapsto R_{t:t+H}$.
- `test()` saves `test_pred_{H}.npy` of predicted residuals E in raw residual units. That file is `file2_path` in stage 5.

Use a different `--model_id` for this run, or it will overwrite the base model’s files in `predictions/<model_id>/.`

The loss: the paper trains f_ϕ with Huber loss. In the code, `_select_criterion()` returns `nn.MSELoss()` whatever `--loss` says (see Section 9).

8 Stage 5: correction and scoring

`final_performance.py` loads three arrays: base test predictions `series1` (N, H, C), base test truth, and residual predictions `series2` (M, H, C) with $M < N$.

8.1 Alignment

The residual series starts L rows later and has its own 20% test split, so it has fewer test windows. Both test splits end on the last row of the data, so the script drops the first $N - M$ base windows. Window k of the trimmed base array then forecasts the same H timestamps as residual window k . (The inline comment says `N - M = seq_len`. In general it is the difference in the rounded 20% test sizes, but the code uses the actual difference, so the alignment is still correct.)

8.2 Correction

```
final_preds = series1_aligned - 0.05 * series2
```

This happens in raw units, as the paper’s “magnitude divergence” argument requires. The script prints raw-scale MSE/MAE for the base model (“Before”) and the corrected forecast (“After”), and saves `final_preds_original_scale.npy`.

8.3 Re-scaling for the reported numbers

Last, it fits a new `StandardScaler` on `ground_truth_aligned.flatten()`, which is *all test values of all channels pooled into one column*. It transforms truth and predictions with it and prints “Final MSE/MAE after scaling”. These are the numbers that go into the paper’s tables. The next section explains why they are not the same metric the baselines report.

9 Where the code and the paper differ

These come from reading the code against the paper. None of them were run, because the repository ships no datasets or checkpoints. They are ordered by how much they affect the results.

HIGH Reported “scaled” metric does not match the baselines’ metric

`final_performnace.py:59-85`

Published LSTF numbers, including iTransformer’s, compute error on data standardised *per channel* using train-split statistics:

$$\text{MSE}_{\text{bench}} = \frac{1}{NHC} \sum_{n,h,c} \left(\frac{\hat{y}_{nhc} - y_{nhc}}{\sigma_c^{\text{train}}} \right)^2.$$

The script instead uses one global σ computed over the *test* ground truth of every channel at once (the mean cancels in the difference):

$$\text{MSE}_{\text{code}} = \frac{1}{NHC} \sum_{n,h,c} \left(\frac{\hat{y}_{nhc} - y_{nhc}}{\sigma_{\text{pooled}}^{\text{test}}} \right)^2.$$

When channels have very different scales, σ_{pooled} is dominated by the spread *between* channel levels. Errors in low-magnitude channels then shrink towards zero. ECL mixes clients from tens to thousands of kW, and Weather mixes ~ 1000 mbar pressure with small quantities. On such data the second number can be much smaller than the first even for the same forecast. This fits the unusually flat, very low “Ours” values in Table 1, such as Weather ≈ 0.026 at every horizon, but confirming it needs a re-run. For a like-for-like comparison, rescale with the base run’s train scaler (`train_data.scaler`) per channel. The paper’s Appendix Table 8 (raw-scale errors) is not affected by this issue.

HIGH Huber loss is never used

`run.py:68 · experiments/exp_long_term_forecasting.py:36-38`

`--loss` accepts `MSE | huber | custom2`, but nothing reads `args.loss`, and `_select_criterion()` hard-codes `nn.MSELoss()`. Following the README’s “train using the Huber loss” step as written gives an *MSE*-trained meta-corrector. A minimal fix:

```
def _select_criterion(self):
    if self.args.loss.lower() == 'huber':
        return nn.HuberLoss(delta=1.0)    # delta is not given in the paper
    return nn.MSELoss()
```

The paper does not give δ . PyTorch’s default is 1.0, applied to standardised residuals.

MEDIUM Autoregressive meta-inference is not implemented

`experiments/exp_long_term_forecasting.py:253-352 (active test) · 356-681 (commented out)`

Section 3.2.4 and Fig. 2(g–h) describe building the first residual input from a $2L$ window, then feeding predicted residuals E_j back in. The active `test()` is the standard one-shot sliding-window evaluation. On the residual dataset it reads the *observed* past residuals $R_{t-L:t}$ for every window. This is still causal: each past residual uses only truth and forecasts from before t . But it is a different, and easier, protocol than the one described. The two commented-out “auto-regressive test” functions roll the *base* model forward on its own outputs. They are unrelated to the meta-corrector and disabled.

MEDIUM Training-split residuals come from training-mode passes, not the frozen model

`experiments/exp_long_term_forecasting.py:201-229`

The paper says the base model is frozen and *then* its predictions form R . In the code, `epoch{e}_train_pred` is collected inside the training loop. Dropout is active, and the weights change after every batch, so early windows in the file come from an older model than late ones. You also have to pick by hand the epoch file that matches the checkpoint early stopping kept. Re-running the best checkpoint over the train split in eval mode would match the paper.

MEDIUM α was chosen by counting wins across the benchmark results

`paper App. C.4 · final_performnace.py:12`

$\alpha = 0.05$ is hard-coded. The paper picks it by grid search that counts “wins” across all 32 dataset $\times$ horizon results, and does not say it used validation data. If those were test scores, α was tuned on the test set. The pipeline already saves validation predictions for both models, so α could be tuned on validation instead.

LOW Hard-coded values you must edit per run

`save_as_csv.py · fluctuation_calculation.py · final_performnace.py`

- `save_as_csv.py`: `pred_len = 96` is fixed. For $H \in \{192, 336, 720\}$ you must change it, or windows are mis-merged. `features = 1` must be set to C (321 for ECL).
- `flactuation_calculation.py`: `end = 20000` silently truncates longer datasets (ETTm $\approx$ 69.7k rows, Weather $\approx$ 52.7k, ECL $\approx$ 26.3k). Set `end=None`. `dataset1_path` defaults to a leftover Windows path (G:\MI LAB\BTC\close_price.csv).
- The residual CSV should be trained with `--data custom`. The ETT loaders’ fixed month borders would not line up with the shortened series, and stage 5’s end-alignment assumes both test splits end on the last row.
- File names in the README differ from the repo (`final_performance.py` vs `final_performnace.py`, `fluctuation_calculation.py` vs `flactuation_calculation.py`). Scripts point at `./dataset/...`, which is not in the repo.
- `result_long_term_forecast.txt` and `results/` are keyed by `setting`, which omits the data path. Base and meta runs with the same `model_id` overwrite each other.

10 How to run it (ECL, $H = 96$)

1. **Install and get data.** `pip install -r requirements.txt` (torch 2.1, numpy 1.23, pandas 1.5, sklearn 1.2). Download the benchmark bundle linked in `scripts/multivariate_forecasting/README.md` into `./dataset/`.
2. **Train the base model.** Run the first block of `scripts/multivariate_forecasting/ECL/iTransformer.sh` (3 layers, `d_model=512`, batch 16, `lr  $5 \times 10^{-4}$`). This writes `predictions/ECL_96_96/`.
3. **Merge each split.** Run `save_as_csv.py` three times with `features=321`, `pred_len=96` and the right split file. For train and val, use the epoch that early stopping kept.
4. **Make residuals.** In `flactuation_calculation.py` set `directory_path`, `dataset1_path=./dataset/electricity/electricity.csv`, `dataset='ECL_resid'`, `start, end = 96, None`.

5. **Train the meta-corrector.** Apply the Huber fix from Section 9 first, then run:

```
python -u run.py --is_training 1 --model iTransformer --data custom \
--root_path <dir> --data_path ECL_resid.csv --model_id ECL_resid_96_96 \
--features M --seq_len 96 --pred_len 96 --e_layers 3 \
--enc_in 321 --dec_in 321 --c_out 321 --d_model 512 --d_ff 512 \
--batch_size 16 --learning_rate 0.0005 --loss huber --des Exp --itr 1
```

6. **Combine.** Point `final_performnace.py` at `predictions/ECL_96_96/test_pred_96.npy`, `test_true_96.npy` and `predictions/ECL_resid_96_96/test_pred_96.npy`. Compare “Before” and “After” in raw units. For the benchmark metric, rescale per channel (see Section 9).

The paper reports that the second stage roughly doubles training time ($1.7\text{--}2.0\times$). Inference overhead is $\leq 0.6\%$, because the two forward passes are independent and can run in parallel.

11 Inherited iTransformer code

Much of the repository is upstream iTransformer code that this method does not need: nine other models, the partial-variate training experiment, and four script families for the upstream paper’s ablations (boosting other Transformers, variate generalisation, longer lookbacks, efficient attention). They work as in upstream, with one side effect: the data factory now disables shuffling for training everywhere. The authors made that change on purpose to keep saved predictions in time order, but it also changes the upstream experiments’ training dynamics. `layers/SelfAttention_Family.py` includes a pure-PyTorch block-wise “FlashAttention” (for `iFlashformer`) and the `ProbSparse` and `Reformer LSH` attention (which needs `reformer-pytorch`). None of these are used by iTransformer.

The paper also swaps in GPHT, TimeBridge and TimeBase as base models (Table 2). None of those models are in this repository.